ORGNZ-08 LAB: Grail Segments - Hands-on Exercises

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 8 of 10 | **Type:** LAB | **Created:** February 2026 | **Last Updated:** 04/25/2026

Overview

This lab notebook contains 3 hands-on exercises extracted from **ORGNZ-08: Grail Segments**. Complete the lecture notebook first, then work through these exercises to reinforce the concepts with real DQL queries against your Dynatrace environment.

Table of Contents

Prerequisites

Requirement	Details
Completed	ORGNZ-08: Grail Segments (lecture notebook)
Dynatrace Environment	SaaS tenant with Grail enabled
Permissions	`logs.read`, `metrics.read`, `entities.read`, `spans.read`

Exercise 1: Team-Based Views

ORGNZ-08: Grail Segments

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 8 of 10 | **Created:** January 2026 | **Last Updated:** 04/25/2026

Grail segments are logical groupings of data that enable real-time filtering across massive datasets without creating thousands of individual rules. Segments bring business context to observability data and are consistently available across the Dynatrace platform.

| Requirement | Details |
|---

---

1. Segments vs Buckets
2. Segment Use Cases
3. Segment Structure
4. Segment Limits
5. Segment Design Patterns
6. Leveraging OneAgent Enrichments
7. Entity Relationship Strategies
8. Testing Segments
9. Segments and Access Control
10. Segment Design Checklist

----------|----------|
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	`segment:read` for viewing, `segment:write` for creating
Knowledge	Completed ORGNZ-01 through ORGNZ-07

By the end of this notebook, you will:
- Understand segments vs buckets
- Design effective segment rules
- Use dynamic variables in segments
- Apply entity relationships for comprehensive filtering

Aspect	Buckets	Segments
Purpose	Physical data storage	Logical data filtering
Retention	Controls data retention	No impact on retention
Data types	One type per bucket	Multiple types per segment
Access control	Can be used with IAM	Data filtering only
Cost impact	Direct storage costs	No direct cost impact
Query scope	Storage-level partitioning	Query-time filtering

> **Key insight**: Buckets store data; segments filter it. Use buckets for retention and cost allocation, segments for dynamic data views.

```
Segment: "Platform Team"
Includes: Infrastructure hosts, Kubernetes clusters, platform services
Benefit: Team sees only relevant data, faster troubleshooting
```

```
Segment: "Checkout Application"
Includes: Checkout service, related databases, frontend components
Benefit: Complete application view across logs, traces, metrics
```

```
Segment: "Production Only"
Includes: Entities with environment=production tag
Benefit: Focused production monitoring without dev/test noise
```

```
Segment:
  name: "segment-identifier"
  displayName: "Human Readable Name"
  description: "What this segment filters"

  includes:
    - type: logs
      filter: "host.name starts-with 'prod-'"

    - type: dt.entity.host
      filter: "tags contains 'environment:production'"

    - type: spans
      filter: "service.name == 'checkout'"

  variables:
    - name: environment
      values: ["production", "staging"]
```

Limit	Value	Notes
Maximum includes (rules) per segment	20	Plan rule complexity accordingly
Include blocks per data/entity type	1	Cannot have multiple rules for same type
Expressions per filter condition	10	Maximum conditions per filter
Values per variable	10,000	Maximum variable values

Pattern	Supported?	Notes
`starts-with`	Yes	Must have at least one preceding character
`contains`	No	Not supported in segment filter conditions
`ends-with`	No	Not supported in segment filter conditions

> **Important**: The `contains` restriction applies to **signal data** filters (logs, spans, events). For **entity type** includes, `tags contains 'value'` is valid because it checks tag membership, not substring matching.

```
name: team-platform
displayName: "Platform Team"
description: "All infrastructure and platform services"

includes:
  - type: dt.entity.host
    filter: "tags contains 'team:platform'"

  - type: logs
    filter: "dt.host_group.id starts-with 'platform-'"

  - type: spans
    filter: "service.name starts-with 'platform-'"
```

```
name: app-checkout
displayName: "Checkout Application"
description: "Checkout service and dependencies"

includes:
  - type: dt.entity.service
    filter: "tags contains 'app:checkout'"

  - type: logs
    filter: "k8s.namespace.name == 'checkout'"

  - type: spans
    filter: "service.name starts-with 'checkout-'"
```

```
name: env-production
displayName: "Production Environment"
description: "Production workloads only"

variables:
  - name: env
    values: ["production", "prod"]

includes:
  - type: dt.entity.host
    filter: "tags contains 'environment:${env}'"

  - type: logs
    filter: "dt.host_group.id starts-with 'prod-'"
```

OneAgent automatically enriches signals with metadata for precise filtering:

Enrichment	Source	Example Use
`dt.host_group.id`	Host group configuration	Filter by host group
`k8s.namespace.name`	Kubernetes namespace	Filter by namespace
`k8s.cluster.name`	Kubernetes cluster	Filter by cluster
`cloud.provider`	Cloud detection	Filter by AWS/Azure/GCP
`cloud.region`	Cloud region	Filter by region
`tags`	Entity tags	Filter by custom tags

```
includes:
  - type: logs
    filter: "dt.host_group.id == 'prod-web-tier'"

  - type: dt.entity.host
    filter: "hostGroup == 'prod-web-tier'"
```

Start from hosts and include related entities:

```
includes:
  - type: dt.entity.host
    filter: "tags contains 'team:platform'"

  - type: dt.entity.process_group
    relationship: "runsOn dt.entity.host"

  - type: dt.entity.service
    relationship: "runsOnProcessGroup dt.entity.process_group"
```

Start from services and include supporting infrastructure:

```
includes:
  - type: dt.entity.service
    filter: "tags contains 'app:checkout'"

  - type: dt.entity.process_group
    relationship: "hostsService dt.entity.service"

  - type: dt.entity.host
    relationship: "runsProcessGroup dt.entity.process_group"
```

```
// Test host group filtering (simulating segment rule)
// Note: dt.entity.host does not expose a 'host_group' field in DQL
// Use smartscapeNodes "HOST" which provides the 'hostGroup' field
smartscapeNodes "HOST"
| filter startsWith(hostGroup, "prod-")
| fields name, hostGroup, tags
| limit 20

// Alternative: filter logs directly using the dt.host_group.id signal field
// fetch logs, from:-1h
// | filter startsWith(dt.host_group.id, "prod-")
// | summarize count = count(), by:{dt.host_group.id}
// | sort count desc
```

Exercise 2: Team-Based Views

```
// Test tag-based filtering
fetch dt.entity.service
| expand tags
| filter contains(tags,  "team:platform")
| fields entity.name, tags
| limit 20

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes SERVICE
// | expand tags
// | filter contains(tags,  "team:platform")
// | fields name, tags
// | limit 20
```

Exercise 3: Team-Based Views

```
// Test log filtering by host group
fetch logs, from:-1h
| filter startsWith(host.group,"prod-")
| summarize count = count(), by:{host.group}
| sort count desc
| limit 10
```

Lab Summary

You have completed 3 hands-on exercises for **ORGNZ-08: Grail Segments**.

Exercises Completed

- [ ] Exercise 1: Team-Based Views
- [ ] Exercise 2: Team-Based Views
- [ ] Exercise 3: Team-Based Views

Next Steps

Continue with **ORGNZ-09** for the next notebook.
